CSE 442: SOFTWARE QUALITY ASSURANCE

Assignment 1: Industry Requirements, ID-Personalized Test Engineering & Defect Triage

Scope: Lessons 11 through 14 | Total Marks: 13 Marks

Course Code: CSE 442 (3 Credits)	**Target Focus:** Requirements, Test Cases & Bug Triage
Workload: ~5–7 Hours over 2 Weeks	**Domain:** SkyBound Airways Flight Booking System
Submission Deliverable: Single Excel Sheet	**Personalization:** Student ID Parity (Odd / Even)

1. Industry Scenario & Unstructured Client Transcript

You are a Junior SQA Engineer at **AeroTech Digital Ltd.** You were handed raw meeting notes from a discovery session with the Operations Director of **SkyBound Airways** regarding their new reservation engine (v3.0). Your task is to transform these unstructured client notes into industry-standard SQA artifacts.

[Discovery Meeting Excerpt — SkyBound Airways v3.0]:
"Look, our current flight reservation portal is causing operational chaos during holiday sales. First off, passengers shouldn't be allowed to proceed to checkout if their passport expiration date is within 6 months of the flight date, or if they have an active security hold on their profile. Second, when a flight has open seats—say 180 out of 200 booked—a passenger selecting a seat should hold it for 10 minutes while completing payment, issuing a 'CONFIRMED' ticket upon successful billing. But if the economy cabin hits its maximum seat limit (200/200), don't crash! It should automatically place them on a Standby List. Oh, and the Standby List can't be infinite—cap it at 15 passengers max. If a 16th passenger tries to book, tell them 'Flight & Standby Full'. Finally, on the flight operator dashboard, airline admins need an emergency button to manually cancel a booking or issue a seat override, but that button MUST require a secondary confirmation pop-up with a mandatory reason field so staff don't accidentally cancel a passenger's ticket by misclicking."

2. Task 1: Requirements Engineering & RTM Setup [3 Marks]

Context (Lesson 11): Extract formal software requirements from unstructured client requests.

1. Extract **FOUR (4)** distinct, formal, numbered requirements (**REQ-01** through **REQ-04**) from the client notes above.
2. Map these requirements into a **Requirements Traceability Matrix (RTM)** table on tab **RTM** in your spreadsheet, including: *Requirement ID, Requirement Statement, High-Level Test Scenario ID(s), Test Level (Unit/Integration/System), and Target Execution Status*.

3. Task 2: Test Scenarios & Student ID-Based Personalization [4 Marks]

Context (Lesson 12): Derive test scenarios first, then apply personalized parameters based on your Student ID.

Step A: Test Scenarios derivation — Derive at least 1 high-level Test Scenario (e.g., **TS-01**, **TS-02**...) for each requirement.

Step B: Student ID Personalization Rule (CRITICAL): Check the **LAST DIGIT** of your official University Student ID:

Student ID Last Digit	Assigned Requirements & Scenarios to Detail	Required Test Cases
ODD Digit (1, 3, 5, 7, 9)	Focus on **REQ-01** (Passport/Security Holds) and **REQ-03** (Standby Boundary 200/200 & Cap of 15).	Write **3 Detailed Test Cases** covering Positive, Negative, and Boundary conditions for REQ-01 & REQ-03.
EVEN Digit (0, 2, 4, 6, 8)	Focus on **REQ-02** (10-Min Seat Hold & Confirmation) and **REQ-04** (Admin Emergency Override Pop-up).	Write **3 Detailed Test Cases** covering Positive, Negative, and Boundary conditions for REQ-02 & REQ-04.

*Document all 3 detailed test cases on tab **TestCases**. Fields: ID, Title, Assigned REQ/TS, Preconditions, Test Data, Actions, and Expected Outcome.*

4. Task 3: Defect Engineering & Severity vs. Priority Matrix [4 Marks]

Context (Lessons 13–14): Simulate test case execution failures and draft Jira bug tickets reflecting contrasting Severity and Priority levels.

On tab **DefectReports**, draft **TWO (2)** distinct, formal Jira Bug Reports based on your assigned requirements:

• **Defect Bug Report A — High Severity, Low Priority:** A severe technical flaw (e.g., system memory leak or database deadlock crash) that occurs in a rare, obscure edge case (e.g., entering an unusual special character in middle name at exactly midnight UTC rollover).

• **Defect Bug Report B — Low Severity, High Priority:** A minor functional or visual flaw (e.g., misspelled airline logo 'SkyBoud Airways' or misaligned header text) that is non-destructive but visible to 100% of executive users on the main search page.

Each Bug Report must include: Bug ID, Title, Environment, Steps to Reproduce, Expected vs. Actual Outcome, Severity, Priority, and a 2-sentence business justification.

5. Task 4: Defect Lifecycle & Triage Strategy [2 Marks]

Context (Lessons 13–14): Developer marks Bug Report A as `RESOLVED - FIXED`. You perform re-testing.

On tab **DefectLifecycle**, answer the following two questions:

1. **Re-test Finding:** Upon re-testing, the original crash is resolved, but a new secondary bug is introduced (e.g., the confirmation email fails to dispatch). Explain whether you should **REOPEN** the bug or **CLOSE it & log a NEW Linked Bug**. Detail the state transition.
2. **Developer Communication:** Write a professional, constructive Jira comment to the developer communicating the re-test result.

6. Submission Format & Grading Rubric

Submission Rules: Submit a single Excel / Google Sheet file named **[YourStudentID]_CSE442_Assignment1.xlsx**. Name the four worksheet tabs inside the workbook cleanly without student IDs as follows:

Workbook Tab Name	Task & Criteria Evaluated	Marks
RTM	Task 1: Formal REQ extraction from transcript + RTM setup.	3 Marks
TestCases	Task 2: Test Scenarios + 3 ID-Personalized detailed Test Cases (Odd/Even rule).	4 Marks
DefectReports	Task 3: 2 Jira Bug Reports demonstrating High Sev/Low Pri vs. Low Sev/High Pri.	4 Marks
DefectLifecycle	Task 4: Defect state transition justification + developer Jira comment.	2 Marks
Total	**Industry-Simulated Flight Booking SQA Audit**	**13 Marks**